

Web Security Checklist

Comprehensive checklist based on the OWASP Top 10:2025, MDN Content Security Policy guide, and industry best practices for web application security.

Sources: Items labeled "OWASP A01–A10" are derived from the OWASP Top 10:2025 prevention guidance. Items labeled "MDN CSP" come from the MDN Content Security Policy guide. Items labeled "Beyond articles" are additional recommendations outside the provided articles.

HIGH PRIORITY

Address first. These cover the most exploited vulnerability classes: access control, injection, authentication, cryptography, misconfiguration, and Content Security Policy.

☐	Checklist Item	Source
A01: Broken Access Control		
☐	Deny access by default; require explicit grants for each capability, role, or user	OWASP A01
☐	Implement access control once in a shared module and reuse it across the entire application	OWASP A01
☐	Enforce record ownership in data models — users should only CRUD their own records unless explicitly authorized	OWASP A01
☐	Enforce access control on the server side / API; never rely solely on client-side checks	OWASP A01
☐	Disable web server directory listing and remove .git, backup files, and metadata from web roots	OWASP A01
☐	Log access control failures and alert on repeated failures (potential automated attacks)	OWASP A01
☐	Rate-limit API and controller access to minimize damage from automated attack tooling	OWASP A01
☐	Invalidate stateful session identifiers on the server after logout; keep JWTs short-lived with refresh token rotation	OWASP A01
☐	Protect against CSRF with anti-CSRF tokens, SameSite cookies, or origin header validation	OWASP A01
☐	Prevent CORS misconfiguration: restrict Access-Control-Allow-Origin to trusted domains only	OWASP A01
A02: Security Misconfiguration		

☐	Checklist Item	Source
☐	Create a repeatable, automated hardening process for all environments (dev, QA, prod) with identical config and different credentials	OWASP A02
☐	Remove or disable unused features, components, frameworks, sample apps, documentation, and default accounts	OWASP A02
☐	Review and update configurations as part of every patch management cycle; audit cloud storage permissions (e.g., S3 buckets)	OWASP A02
☐	Send security headers to clients: Strict-Transport-Security, X-Content-Type-Options, X-Frame-Options, Referrer-Policy, Permissions-Policy	OWASP A02
☐	Suppress detailed error messages (stack traces, framework versions) from reaching end users; centralize error interception	OWASP A02
☐	Use segmented architecture: containerization, security groups, or ACLs to separate components and tenants	OWASP A02
☐	Use identity federation and short-lived credentials instead of static keys or secrets embedded in code	OWASP A02
☐	Automate configuration verification; if not automated, manually audit at least annually	OWASP A02
A04: Cryptographic Failures		
☐	Classify all data by sensitivity (PII, financial, health, business secrets) and apply protections per regulatory requirements	OWASP A04
☐	Encrypt all data in transit using TLS 1.2+ with strong cipher suites; enforce HSTS	OWASP A04
☐	Encrypt sensitive data at rest; don't store sensitive data you don't need — discard it ASAP	OWASP A04
☐	Use strong, current algorithms: AES-256 for symmetric encryption; RSA-2048+ or ECC for asymmetric; Argon2id, bcrypt, or scrypt for passwords	OWASP A04
☐	Never use deprecated algorithms: MD5, SHA-1, DES, RC4, or ECB mode	OWASP A04
☐	Never commit cryptographic keys or secrets to source control; store sensitive keys in HSM or cloud key vault	OWASP A04
☐	Implement proper key management: rotation schedules, separate keys per environment, and revocation procedures	OWASP A04
☐	Use authenticated encryption (GCM, CCM) instead of plain encryption modes	OWASP A04

☐	Checklist Item	Source
☐	Use cryptographically secure random number generators (CSPRNG) properly seeded	OWASP A04
A05: Injection		
☐	Use parameterized queries / prepared statements for all database interactions — never concatenate user input into SQL	OWASP A05
☐	Use safe APIs that avoid direct interpreter use, or migrate to ORMs with parameterized interfaces	OWASP A05
☐	Apply positive server-side input validation (allowlists) for all user-supplied data	OWASP A05
☐	Escape output appropriately for the context (HTML, URL, JavaScript, CSS, SQL) to prevent XSS and other injection	OWASP A05
☐	Use LIMIT and other SQL controls to prevent mass data disclosure in case of injection	OWASP A05
☐	Integrate SAST, DAST, and IAST tools into the CI/CD pipeline to catch injection flaws before production	OWASP A05
A07: Authentication Failures		
☐	Implement and enforce multi-factor authentication (MFA) for all users, especially privileged accounts	OWASP A07
☐	Never ship or deploy with default credentials; change all default passwords before deployment	OWASP A07
☐	Check new and changed passwords against the top 10,000 worst passwords list and known-breached credential databases	OWASP A07
☐	Align password policies with NIST 800-63b: favor length over complexity; don't force rotation unless breach is suspected	OWASP A07
☐	Harden login, registration, and password recovery against account enumeration (use identical error messages for all outcomes)	OWASP A07
☐	Limit and progressively delay failed login attempts; log all failures and alert on credential stuffing patterns	OWASP A07
☐	Generate new high-entropy session IDs after login; store in secure, HttpOnly, SameSite cookies; invalidate on logout and timeout	OWASP A07
Content Security Policy (CSP)		
☐	Deploy a Content-Security-Policy header on all responses, not just the main document	MDN CSP
☐	Restrict script-src to prevent inline scripts and eval(); use nonces or hashes for legitimate inline scripts	MDN CSP

☐	Checklist Item	Source
☐	Set default-src to 'none' or 'self' and explicitly allowlist only needed resource origins	MDN CSP
☐	Use frame-ancestors directive (replaces X-Frame-Options) to prevent clickjacking	MDN CSP
☐	Enable upgrade-insecure-requests to automatically upgrade HTTP resources to HTTPS	MDN CSP
☐	Use CSP report-uri or report-to to collect violation reports and iterate on your policy	MDN CSP
☐	Start with Content-Security-Policy-Report-Only to test policies before enforcing them	MDN CSP

MEDIUM PRIORITY

Tackle once high-priority items are in place. Covers supply chain, secure design, integrity verification, logging & alerting, error handling, and HTTP security headers.

☐	Checklist Item	Source
A03: Software Supply Chain Failures		
☐	Generate and maintain a Software Bill of Materials (SBOM) for your entire application, including transitive dependencies	OWASP A03
☐	Continuously monitor dependencies for known vulnerabilities using tools like Dependabot, Snyk, OWASP Dependency-Check, or retire.js	OWASP A03
☐	Only obtain packages from official, trusted sources over secure links; prefer signed packages	OWASP A03
☐	Remove unused dependencies, features, components, files, and documentation to reduce attack surface	OWASP A03
☐	Patch vulnerable dependencies in a risk-based, timely fashion — don't batch patches into monthly or quarterly cycles	OWASP A03
☐	Track and document changes across your entire supply chain: IDEs, extensions, repos, build tools, image registries	OWASP A03
☐	Apply least-privilege access controls and separation of duty across the supply chain — no single person should push code to production unchecked	OWASP A03
☐	Ensure CI/CD pipeline security is at least as strong as the systems it builds and deploys	OWASP A03
☐	Monitor libraries and components that are unmaintained or end-of-life and plan migration	OWASP A03
A06: Insecure Design		
☐	Perform threat modeling for authentication flows, access control, business logic, and key data flows	OWASP A06
☐	Integrate security requirements into user stories and acceptance criteria from the start	OWASP A06
☐	Use secure design patterns and reference architectures; build a library of reusable secure components	OWASP A06
☐	Implement rate limiting and resource controls at the design level for APIs, file uploads, and purchases	OWASP A06
☐	Separate tenant access securely in multi-tenant architectures at the design level	OWASP A06

☐	Checklist Item	Source
☐	Write unit and integration tests for all critical security flows, not just happy-path scenarios	OWASP A06
☐	Apply defense-in-depth: layer controls so a single failure doesn't result in full compromise	OWASP A06
A08: Software or Data Integrity Failures		
☐	Verify integrity of all software updates, libraries, and dependencies using digital signatures or checksums	OWASP A08
☐	Only consume libraries from trusted repositories; consider an internal vetted mirror for high-risk profiles	OWASP A08
☐	Require code review for all changes — no code or config change should reach production without peer review	OWASP A08
☐	Ensure CI/CD pipeline has proper segregation, configuration, and access control for build integrity	OWASP A08
☐	Never accept unsigned or unverified serialized data from untrusted clients; validate integrity before processing	OWASP A08
☐	Use Subresource Integrity (SRI) attributes for all third-party scripts and stylesheets loaded from CDNs	OWASP A08
A09: Security Logging & Alerting Failures		
☐	Log all logins (success and failure), access control failures, and server-side input validation failures with sufficient user context	OWASP A09
☐	Ensure every security control in the app logs whether it succeeds or fails	OWASP A09
☐	Format logs for easy consumption by SIEM or log management tools	OWASP A09
☐	Encode log data properly to prevent log injection attacks	OWASP A09
☐	Protect log integrity with append-only storage or tamper-evident mechanisms	OWASP A09
☐	Set up alerting thresholds and escalation processes; ensure alerts are reviewed within a defined SLA	OWASP A09
☐	Verify that penetration testing and DAST scans trigger your alerting systems	OWASP A09
☐	Never log sensitive data (passwords, tokens, PII, PHI) — mask or redact before logging	OWASP A09
☐	Back up logs to a centralized, remote location — don't rely on local storage only	OWASP A09
A10: Mishandling of Exceptional Conditions		

☐	Checklist Item	Source
☐	Catch and handle errors at the point where they occur, not just at a global level	OWASP A10
☐	Always fail closed: roll back partial transactions fully and start over rather than attempting recovery mid-transaction	OWASP A10
☐	Never expose sensitive information (stack traces, internal paths, SQL queries) in error messages shown to users	OWASP A10
☐	Implement a global exception handler as a safety net for uncaught errors	OWASP A10
☐	Add rate limiting, resource quotas, and throttling to prevent exceptional conditions from being triggered by attackers	OWASP A10
☐	Log all errors and exceptions with enough context for forensic analysis; issue alerts for patterns indicating attack	OWASP A10
☐	Handle missing parameters, null values, insufficient privileges, and resource exhaustion explicitly rather than letting them propagate	OWASP A10
HTTP Security Headers (beyond CSP)		
☐	Set Strict-Transport-Security with a long max-age (e.g., 31536000), includeSubDomains, and consider preload	Beyond articles
☐	Set X-Content-Type-Options: nosniff to prevent MIME-type sniffing	Beyond articles
☐	Set Referrer-Policy to strict-origin-when-cross-origin or no-referrer	Beyond articles
☐	Set Permissions-Policy to disable unused browser features (camera, microphone, geolocation, payment, etc.)	Beyond articles
☐	Remove server version headers (Server, X-Powered-By) that reveal tech stack information	Beyond articles
☐	Set secure cookie attributes on all cookies: Secure, HttpOnly, SameSite=Lax or Strict, appropriate Path and Domain	Beyond articles

LOW PRIORITY

Valuable for mature security programs. Covers API hardening, client-side defense, infrastructure/DevOps, incident response, and privacy/data protection.

☐	Checklist Item	Source
API-Specific Security		
☐	Implement authentication and authorization on every API endpoint — don't assume internal APIs are safe	Beyond articles
☐	Validate and sanitize all API input including headers, query params, body, and path parameters	Beyond articles
☐	Use API rate limiting and throttling per-user and per-IP to prevent abuse	Beyond articles
☐	Version your APIs and deprecate insecure older versions with clear sunset timelines	Beyond articles
☐	Implement request/response schema validation (e.g., OpenAPI spec enforcement)	Beyond articles
☐	Avoid exposing internal IDs or sequential identifiers; use UUIDs or opaque tokens	Beyond articles
Client-Side Security		
☐	Sanitize all user-generated content rendered in the DOM using a trusted library (DOMPurify or equivalent)	Beyond articles
☐	Avoid using innerHTML, document.write, eval(), and other dangerous DOM APIs with user-controlled data	Beyond articles
☐	Implement Trusted Types CSP policy to prevent DOM-based XSS at the browser level	Beyond articles
☐	Use Subresource Integrity (SRI) for all third-party scripts and stylesheets	Beyond articles
☐	Audit and minimize third-party JavaScript: analytics, ads, widgets, and tag managers expand the attack surface	Beyond articles
Infrastructure & DevOps Security		
☐	Scan container images for vulnerabilities before deployment; use minimal base images	Beyond articles
☐	Run application processes with minimal privileges (non-root containers, least-privilege IAM roles)	Beyond articles
☐	Implement network segmentation: separate database, application, and public-facing tiers	Beyond articles
☐	Use infrastructure-as-code with security policies checked in and version-controlled	Beyond articles

☐	Checklist Item	Source
☐	Rotate secrets (API keys, database passwords, tokens) automatically on a schedule	Beyond articles
☐	Enable cloud provider security features: VPC flow logs, GuardDuty/Security Center, WAF	Beyond articles
Incident Response & Organizational		
☐	Create and test an incident response plan with defined roles, communication channels, and escalation procedures	Beyond articles
☐	Conduct regular security training for all developers covering OWASP Top 10 and secure coding practices	Beyond articles
☐	Perform regular penetration testing (at least annually, or per major release)	Beyond articles
☐	Run a responsible disclosure or bug bounty program	Beyond articles
☐	Maintain an asset inventory of all public-facing applications, APIs, and their dependencies	Beyond articles
☐	Document and review your security architecture at least annually	Beyond articles
Privacy & Data Protection		
☐	Implement data minimization: only collect and retain the data you actually need	Beyond articles
☐	Honor data deletion requests and implement automated data retention/purging policies	Beyond articles
☐	Anonymize or pseudonymize personal data in non-production environments	Beyond articles
☐	Ensure compliance with applicable privacy regulations (GDPR, CCPA, HIPAA) for your data handling	Beyond articles